

MetroPulse

Real-Time Public Transport Tracking & Management System

Comprehensive Project Documentation & UML Architecture Specification

PROJECT METADATA & SYSTEM SPECIFICATIONS

Domain: Smart Mobility & Fleet Telemetry

Architecture: MERN Stack + React Native + WebSockets (Socket.IO)

Geospatial Engine: OpenStreetMap + Leaflet + GeoJSON 2dsphere

Security: httpOnly Cookie Auth + Role-Based Access Control (RBAC)

Date: October 2026

1. EXECUTIVE SUMMARY & ARCHITECTURE OVERVIEW

1.1 Executive Summary

MetroPulse is an enterprise-grade, real-time public transit tracking and fleet management platform designed to eliminate passenger wait-time uncertainty. It seamlessly connects bus drivers broadcasting live GPS telemetry, passengers tracking transit routes with automated ETAs, and transit administrators overseeing fleet operations.

1.2 Key Objectives

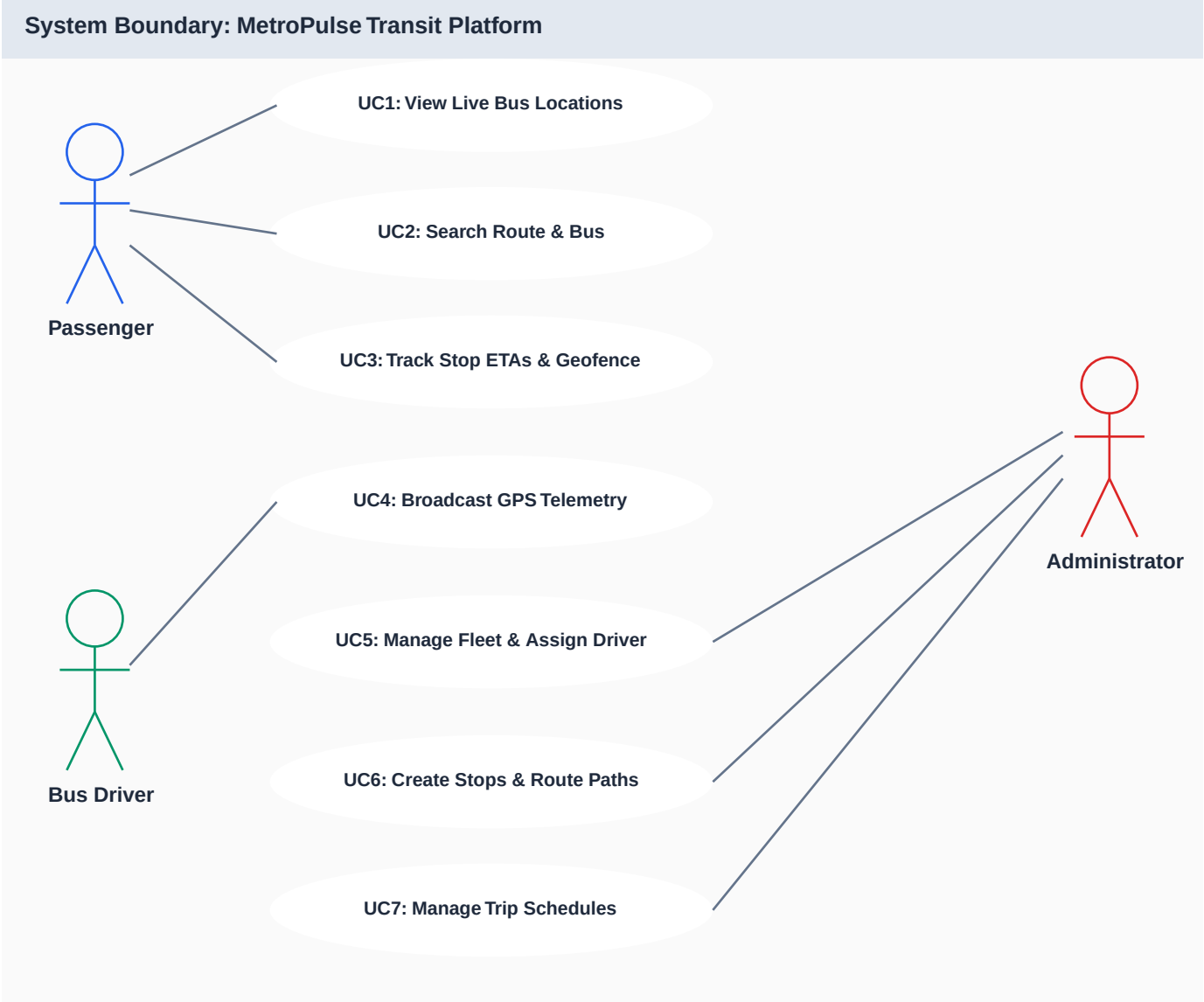
- Real-Time Telemetry: High-frequency GPS updates pushed via WebSockets (Socket.IO) with sub-second latency.
- Dynamic Geofencing & ETAs: Automated 200-meter arrival detection and traffic-adjusted arrival estimation.
- Zero-Cost Open Mapping: Complete elimination of commercial mapping API keys using OpenStreetMap & Leaflet.
- Enterprise Security: httpOnly cross-origin cookie authentication protecting token exchange.
- Multi-Platform Reach: Responsive Passenger Portal, Admin Dashboard, and Driver Mobile Application.

1.3 Technology Stack Specifications

Tier	Technology	Role / Capability
Passenger Web	React 19, Vite, Leaflet	Live interactive transit map, route search, ETA display
Admin Web	React 19, Tailwind v4, Axios	Fleet, route, stop, schedule, and driver administration
Mobile App	React Native, Expo Location	Driver background GPS tracking and live broadcast
Backend API	Node.js, Express.js (ESM)	RESTful endpoints, JWT & cookie auth, business logic
Real-Time Server	Socket.IO Engine	Bi-directional location streaming & event rooms
Database	MongoDB Atlas (Mongoose)	Spatial 2dsphere indexing, location history, schemas

2. UML USE CASE DIAGRAM

System Actors & Use Case Interactions

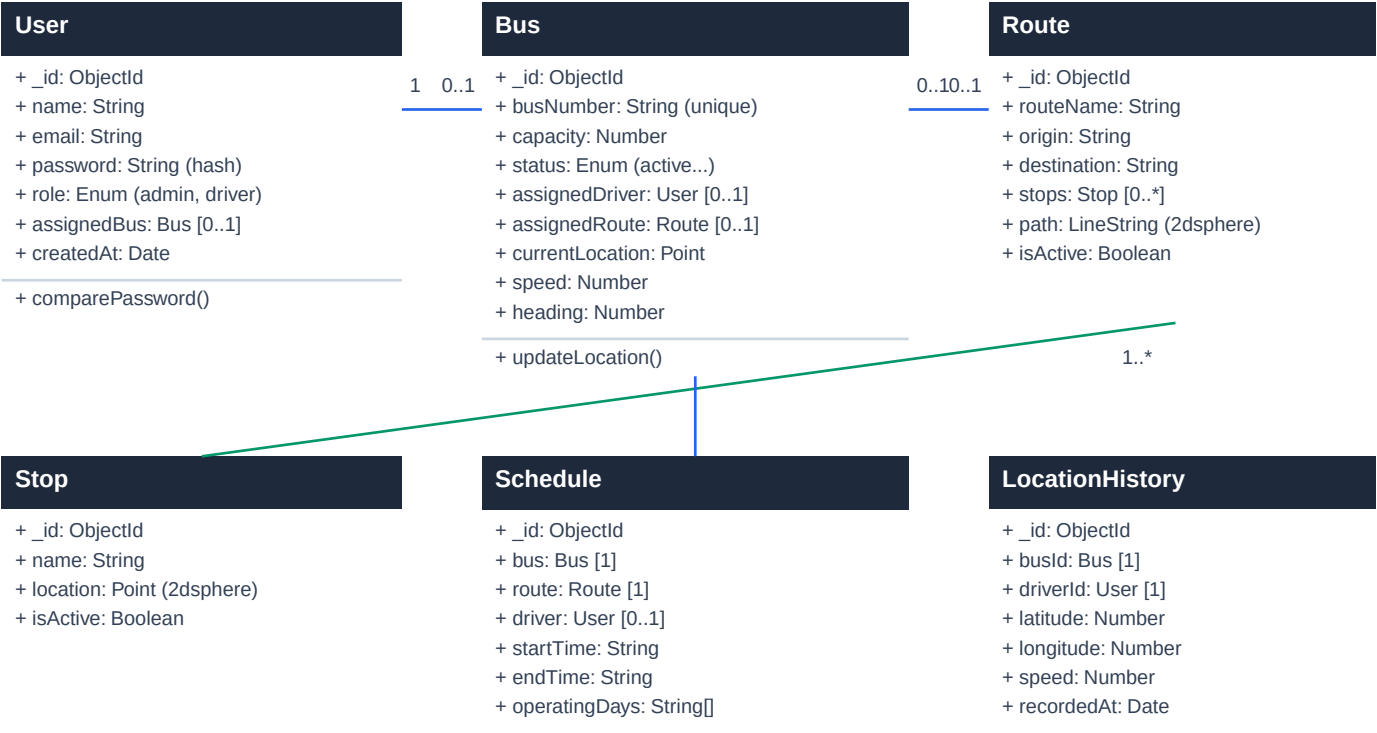


2.1 Use Case Specifications

- UC1 & UC2 (Passenger): Real-time map rendering and unified search by bus number or route name.
- UC3 (Passenger): Calculates stop-by-stop ETAs and triggers arrival indicators when within 200m geofence.
- UC4 (Driver): Background GPS polling transmits coordinates, speed, and heading to Socket.IO backend.
- UC5 - UC7 (Admin): Full CRUD lifecycle management for transit assets with spatial validation.

3. UML CLASS DIAGRAM & DATA ARCHITECTURE

Domain Model Classes & Schema Relationships

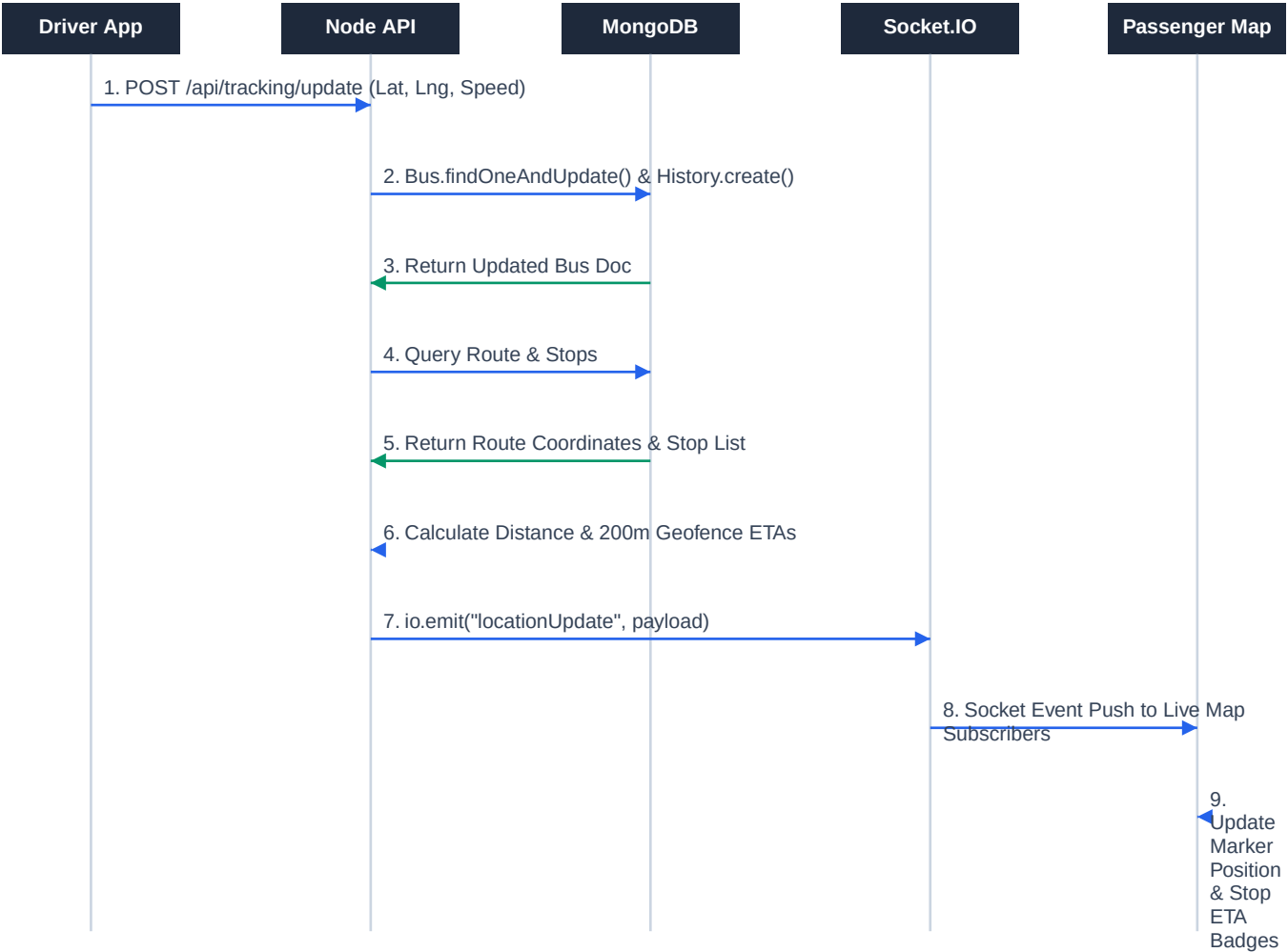


3.1 Geospatial Indexing Rules

- Stop Location: Indexed using 2dsphere [longitude, latitude] GeoJSON Point.
- Route Path: Indexed using 2dsphere GeoJSON LineString with sparse: true index. Prevents index failure when path coordinates are empty or < 2 points.
- Auto Path Construction: Backend automatically constructs LineString coordinates from ordered Route Stops.

4. UML SEQUENCE DIAGRAM (LIVE GPS TELEMETRY)

Real-Time Location Broadcast & ETA Pipeline Flow



4.1 Telemetry Performance Guarantees

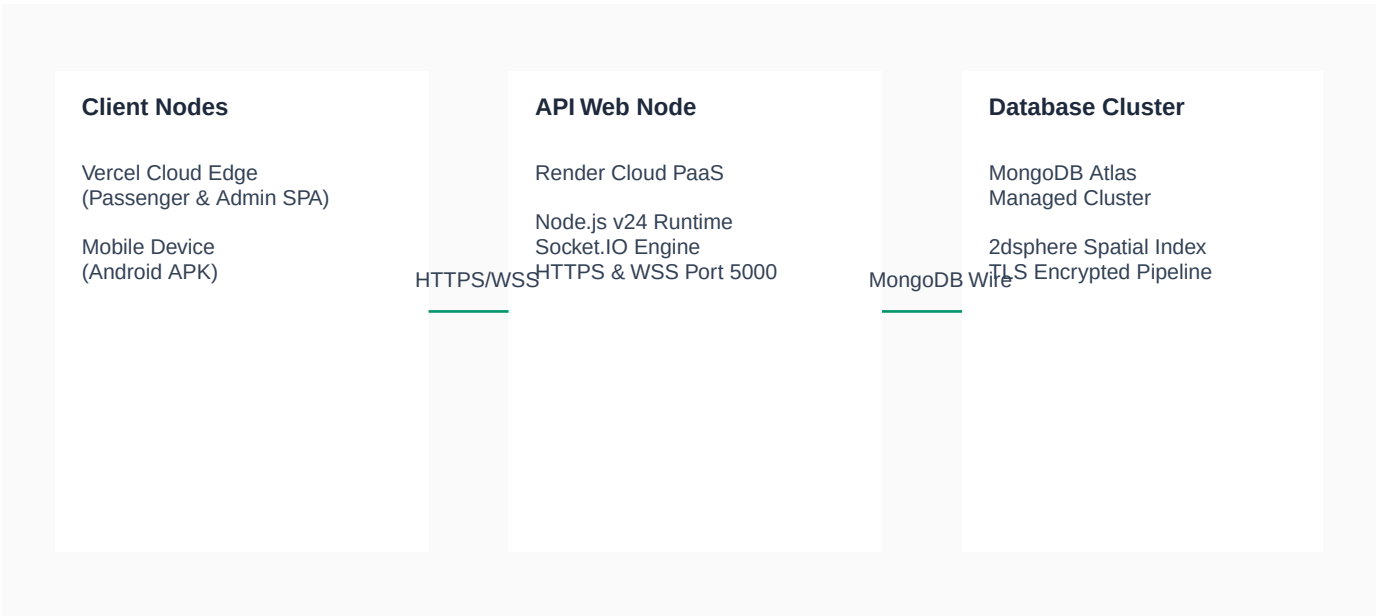
- Sub-second Latency: Broadcasts update payload over established WebSocket channels in < 50ms.
- Driver Auth Isolation: Driver role enforces bus resolution strictly from JWT claims (prevents bus ID spoofing).
- Automatic Fallback: If bus speed is 0 km/h (traffic/stop), ETA calculator falls back to 15 km/h city average.

5. UML COMPONENT & DEPLOYMENT DIAGRAMS

5.1 Component Diagram



5.2 Deployment Topology Diagram



6. Production Deployment Summary

- API Production Endpoint: <https://ruler-transport-tracker-backend.onrender.com/api>
- Admin Default Credentials: `admin@bustracker.com` / `AdminPassword123!`
- Passenger & Admin Hosting: Vercel Single Page Application Rules (`vercel.json`)

